Cuaderno de datos

La base de BE, de lo macro a lo micro

Se lee después de la guía central. Empieza por lo que es una base de datos y dónde está la de BE, sigue por cada área del modelo y termina en lo micro: claves, disparadores, transacciones, índices, historia, migraciones y unas consultas de lectura.

Conceptual y físico. Los diagramas por área muestran solo las tablas centrales, con sus nombres y relaciones reales: lo que se omite se nombra en el texto. El esquema completo son 118 tablas; las 15 vistas del DER de la tesis las cubren todas, y su definición (`laminas/motor/vistas-der.mjs`) se puso al día acá con Administración.

Commit de referencia: `e4512cb` (`main`, 2026-10-26).

De dónde sale cada dato: `prisma/schema.prisma` y `prisma/migrations`, leídos por herramientas (`herramientas/leer-esquema.mjs`, `clasificar-tablas.mjs`, `restricciones-de-la-base.mjs`). Nada se copió a mano.

Datos de los ejemplos: sintéticos, los de las cuentas «Prueba» de la prueba breve.

CONTENIDO

Parte 1 · Lo macro: qué es la base y dónde está

¿Qué es una base de datos, y cómo se anidan sus piezas? 2
¿Cuántas bases tiene BE, y dónde está cada una? 3
¿Qué hace Prisma, y qué no hace? 4
¿Por qué hay tantas tablas? 5

Parte 2 · Las áreas, una por una

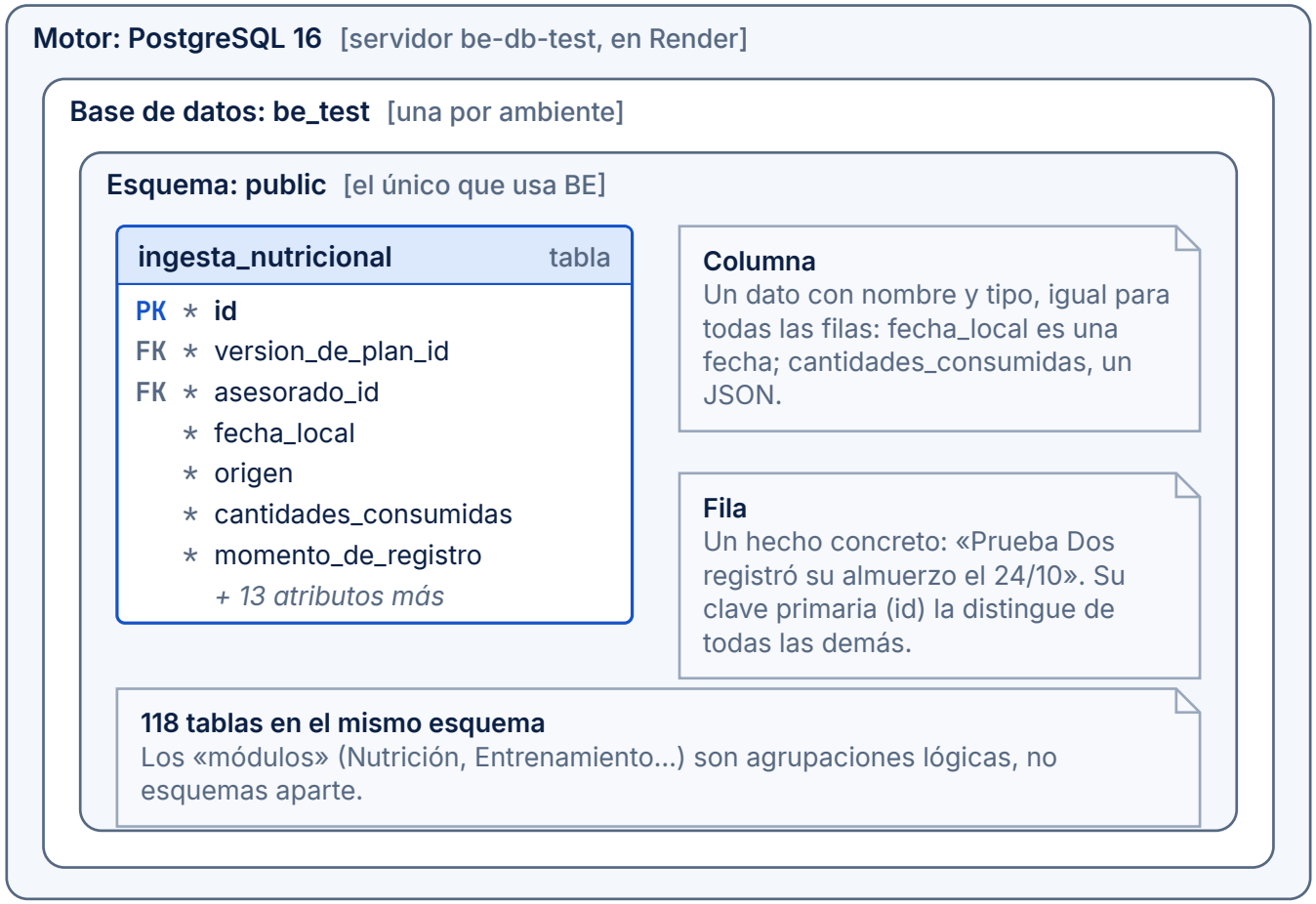
Identidad, sesiones y Administración: ¿quién es quién? 6
Vínculos y consentimientos: ¿cómo decide la persona quién accede? 7
Nutrición: ¿cómo se guarda un plan sin que cambie después? 8
Entrenamiento: ¿qué se registra de una sesión? 9
Antropometría: ¿por qué acá el resultado sí se guarda? 10
Objetivos, procesos y revisiones: ¿cómo se ordena el seguimiento? 11
Auditoría, medios e idempotencia: ¿qué guarda BE sobre su propio funcionamiento? 12

Parte 3 · Lo micro: claves, reglas, transacciones y consultas

¿Cuándo una relación necesita su propia tabla? 13
¿Qué impide que los datos queden inconsistentes? 14
¿Qué pasa si algo falla a la mitad, o si dos pedidos llegan a la vez? 15
¿Para qué sirven los índices, y cómo guarda BE la historia? 16
¿Cómo se juntan dos tablas en una consulta? 17
¿Cómo se cuenta y se agrupa, sin inventar ceros? 18
¿En qué se diferencia el modelo conceptual del físico? 19

¿Qué es una base de datos, y cómo se anidan sus piezas?

Un motor guarda bases; una base tiene esquemas; un esquema, tablas; una tabla, columnas y filas.



CONCEPTO 1

Sistema gestor de bases de datos (SGBD)

En criollo. El programa que guarda los datos y hace cumplir sus reglas, aunque el programa que los usa se equivoque.
En BE. PostgreSQL 16. En test, el servicio `be-db-test` de Render; en la CI, un contenedor que se descarta.
Para comprobar. Si la API tuviera un error y quisiera borrar una comida registrada, ¿quién la frena?

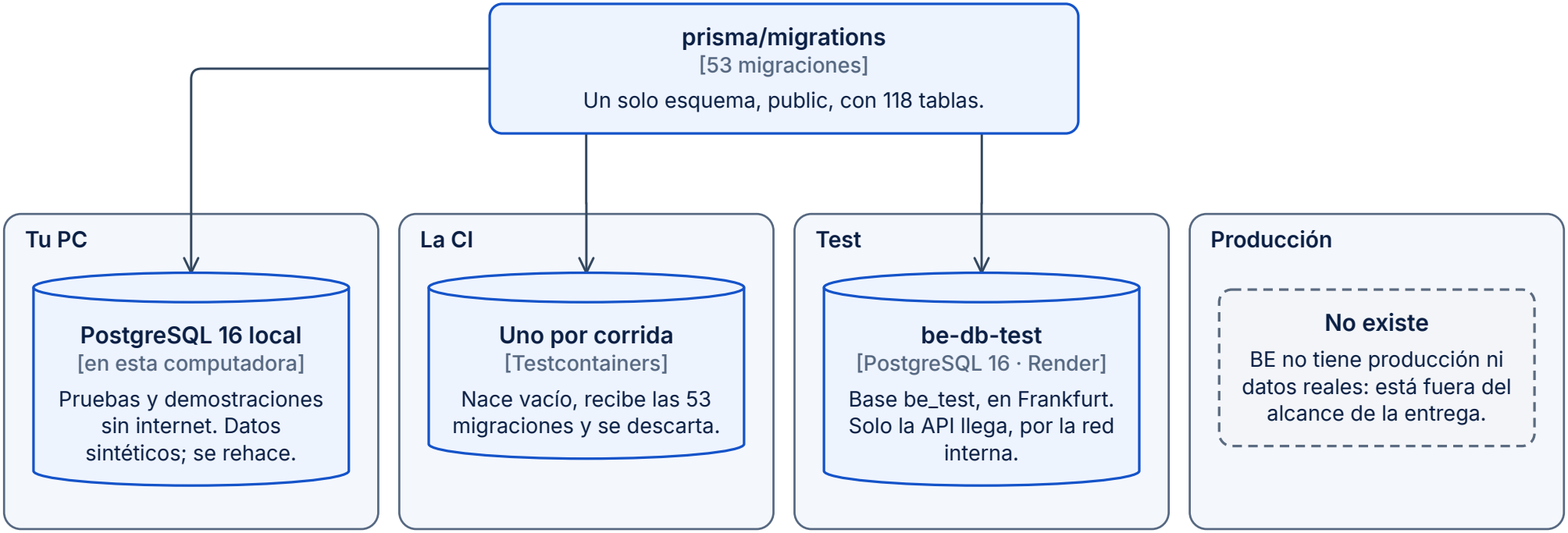
CONCEPTO 2

Esquema

En criollo. Una carpeta con nombre dentro de la base, que agrupa tablas.
En BE. BE usa uno solo, `public`, con sus 118 tablas. Las áreas (Nutrición, Entrenamiento…) son agrupaciones lógicas del código, no esquemas aparte.
Para comprobar. ¿Qué ganaría y qué perdería BE si cada área tuviera su esquema?

¿Cuántas bases tiene BE, y dónde está cada una?

Una por ambiente, y las tres con el mismo esquema. Producción no existe: no hay datos reales.



CONCEPTO 3

Ambiente

En criollo. Una copia entera del sistema (programas, base y configuración) para un uso: probar, mostrar o atender gente de verdad.
En BE. Tres: tu PC, la CI y test. Test tiene la cohorte sintética y los escenarios «Prueba». Una prueba en la CI nunca toca la base de test, y una prueba local se niega a correr contra una base que no sea local.
Para comprobar. ¿Por qué conviene que una prueba automática nunca pueda conectarse a la base de test?

CONCEPTO 4

Módulo no es ambiente

En criollo. Un módulo es una parte del código (un área); un ambiente es dónde corre todo junto.
En BE. Nutrición, Entrenamiento y Antropometría son módulos: sus tablas conviven en el mismo esquema `public` de cada base. No hay «la base de Nutrición».
Para comprobar. Si mañana se agrega un área, ¿cuántas bases nuevas hacen falta?

¿Qué hace Prisma, y qué no hace?

Prisma traduce el modelo escrito en el código a tablas, genera el cliente con el que la API consulta y aplica las migraciones. Las reglas duras las pone la base.

Un fragmento real de `prisma/schema.prisma`, recortado:

```
/// T-06-54 — Ingesta registrada: hecho confirmado contra una versión ACTIVADA; nunca
/// modifica la prescripción (REG-06-106). … Append-only.
model IngestaNutricional {
  id              String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  versionDePlanId String   @map("version_de_plan_id") @db.Uuid
  asesoradoId     String   @map("asesorado_id") @db.Uuid
  origen          OrigenDeIngesta
  fechaLocal      DateTime @map("fecha_local") @db.Date
  cantidadesConsumidas Json? @map("cantidades_consumidas")

  versionDePlan VersionDePlanNutricional @relation(fields: [versionDePlanId],
                references: [id], onDelete: Restrict, onUpdate: Restrict)

  @@index([asesoradoId, fechaLocal])
  @@map("ingesta_nutricional")
}
```

Lo que Prisma hace	Lo que no hace
Convierte cada `model` en una tabla y cada campo en una columna con su tipo.	No hace cumplir «append-only» ni las transiciones de estado: eso lo hacen **disparadores** escritos en SQL dentro de las migraciones.
Genera el cliente tipado: la API no puede pedir una columna que no existe.	No decide quién puede leer qué: eso es el PDP, en la API (guía central, nivel 3).
Guarda la historia de migraciones aplicadas en la tabla `_prisma_migrations`.	No inventa los datos que faltan: un campo opcional (`?`) queda en `NULL`.

CONCEPTO 5

ORM (mapeo objeto-relacional)

En criollo. La capa que deja trabajar con objetos del lenguaje en lugar de escribir SQL a mano.
En BE. El cliente de Prisma: la API escribe `tx.ingestaNutricional.create(…)` y Prisma lo convierte en un `INSERT INTO ingesta_nutricional …`.
Para comprobar. Si Prisma arma el SQL, ¿por qué BE igual tiene SQL escrito a mano en las migraciones?

CONCEPTO 6

Mapeo de nombres

En criollo. El código usa un nombre; la base, otro. Una línea dice cuál corresponde a cuál.
En BE. `fechaLocal` en el código es `fecha_local` en la tabla (`@map`); el modelo `IngestaNutricional` es la tabla `ingesta_nutricional` (`@@map`).
Para comprobar. ¿Dónde mirarías para saber cómo se llama en la base el campo `versionDePlanId`?

¿Por qué hay tantas tablas?

118 tablas no son 118 cosas distintas: más de un tercio existe para que nada se pise y todo tenga historia.

Tablas	Para qué existen	Ejemplos
37	Cosas del negocio que existen y cambian de estado	Identidad, Plan nutricional, Ingesta, Proceso operativo
17	Hechos: lo que pasó, en orden, sin cambiar nunca	Evento de nutrición, Decisión de acceso, Registro de auditoría
16	Versiones e instantáneas: cambiar sin pisar	Versión del plan, Instantánea del plan, Versión de consentimiento
16	Operación técnica: sesiones, seguridad, idempotencia	Sesión, Factor TOTP, Registro de idempotencia, Medio
12	Catálogos y textos de referencia	Elemento del catálogo nutricional, Ejercicio, Receta
10	Correcciones, anulaciones y retiros	Rectificación de cantidades, Anulación de ingesta, Corrección de medición
10	Relaciones y asociaciones	Vínculo, Alcance del vínculo, Destinatario de una nota
118	**Total**, en un solo esquema	La lista completa por clase: `datos/clases-de-tablas.json`

Lo necesario. Hechos, versiones y correcciones (43 tablas) salen de tres reglas del legajo: la historia no se borra, un plan activado no se edita y corregir es agregar. Sacarlas sería sacar esas garantías.

Lo discutible. Nutrición y Entrenamiento repiten la misma forma (objetivo, versión, plan, versión, instantánea, registro, corrección, revisión, eventos): es más código para mantener, a cambio de áreas que no se mezclan. Cada área tiene además su propia tabla de eventos. Son decisiones conscientes, pero son la primera candidata a «deuda» si BE sumara una cuarta área.

CONCEPTO 7

Complejidad necesaria y deuda

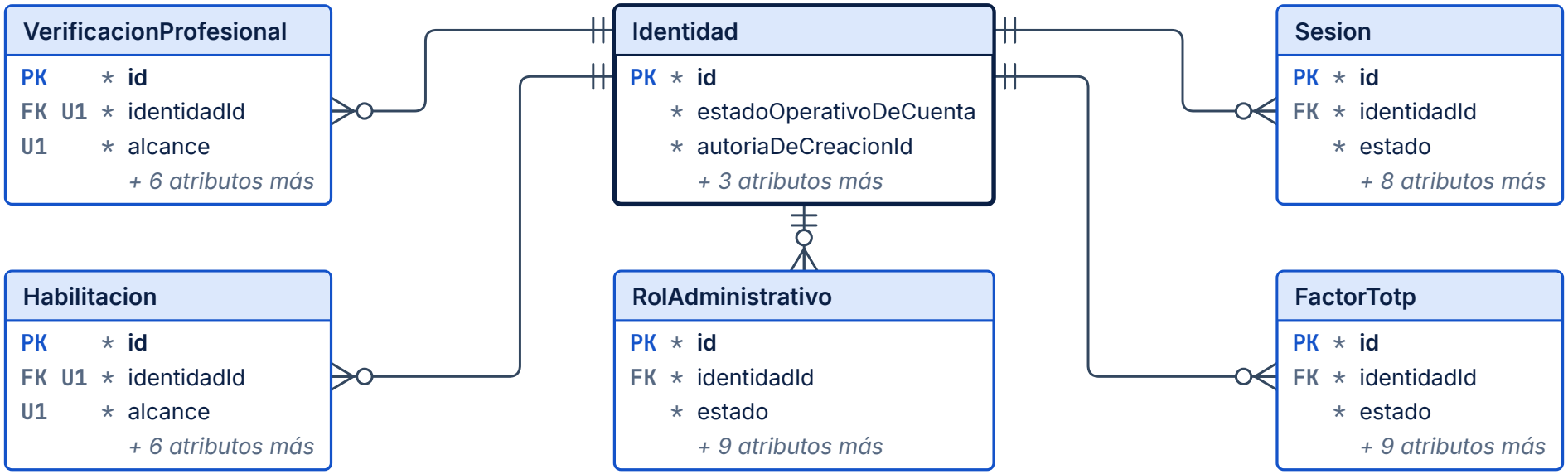
En criollo. Necesaria es la que exige el problema. Deuda es la que agregamos sin que el problema la pida, y se paga después.
En BE. Que corregir no borre exige tablas de corrección: es necesaria. Tener dos estructuras casi iguales para los planes de Nutrición y de Entrenamiento es una decisión discutible.
Para comprobar. Nombrá una tabla que juntarías con otra, y qué regla del legajo tendrías que romper para hacerlo.

No se fusionan para bajar el número

Unir tablas para que «sean menos» suele mezclar hechos con estados y obliga a actualizar filas que hoy no se tocan. La pregunta útil no es «¿cuántas?», sino «¿cuál de estas no responde a una regla?».

Identidad, sesiones y Administración: ¿quién es quién?

Identidad, sesiones y Administración Una identidad por persona; sesiones revocables; la situación de cada profesional, por área.



uno y solo uno · cero o uno · cero o muchos · PK FK claves · * obligatorio · o opcional · ↗ clave hacia otra área

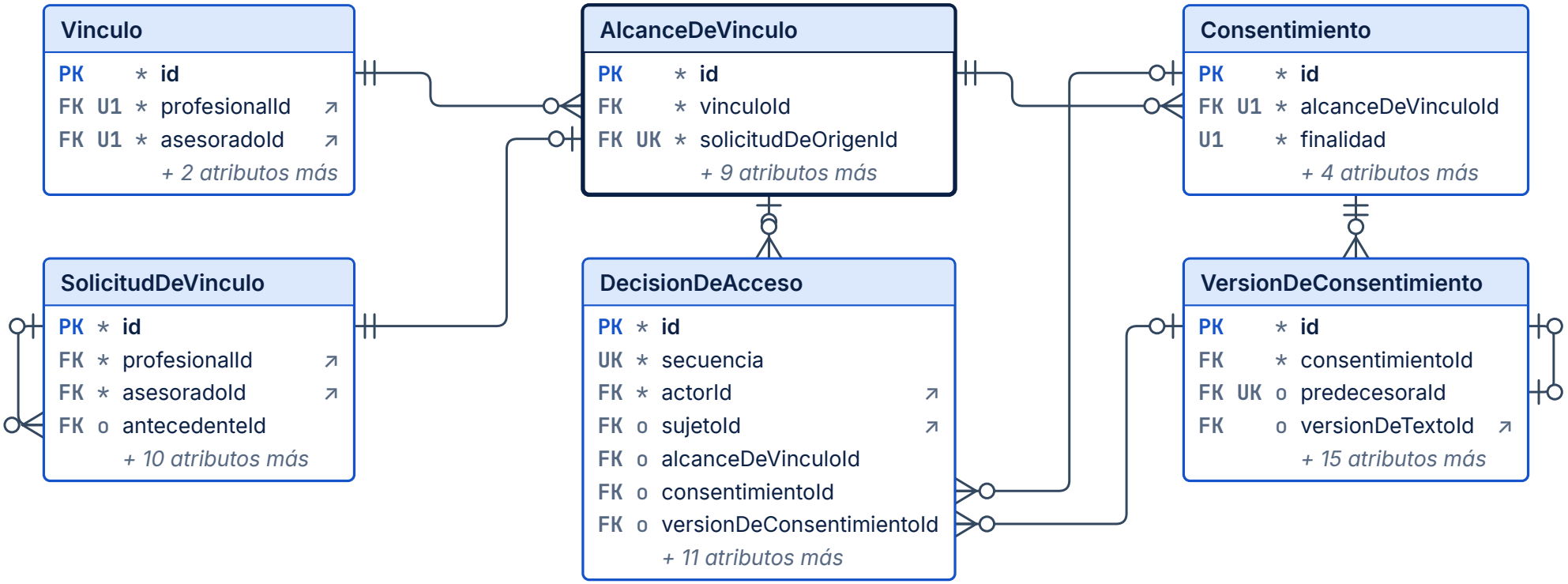
Qué representa. `identidad`: una persona, un profesional o un administrador; clave primaria `id` (UUID), como casi todas. `sesion`: cada inicio de sesión; el token no se guarda, solo su id. `verificacion_profesional` y `habilitacion`: la situación de un profesional en un área, separadas. `rol_administrativo` y `factor_totp`: el rol de Administración y su autenticador.

Guardado o calculado. Se guarda el estado de cada sesión y su vencimiento; «puede operar el área» se calcula con la verificación y la habilitación. El secreto del autenticador va cifrado (AES-256-GCM); los códigos de recuperación y de incorporación, solo como huella.

Qué cambia y qué se conserva. La verificación cambia de estado y cada paso queda en `evento_de_verificacion`. Una sesión no se borra: se revoca o vence. Un rol administrativo no se elimina. Fuera del diagrama: `metodo_de_acceso` (contraseña o Google), `perfil_profesional`, `control_de_sesion` y el segundo factor.

Vínculos y consentimientos: ¿cómo decide la persona quién accede?

Vínculos y consentimientos Un vínculo por pareja, un alcance por área, una autorización por alcance y una decisión por pedido.



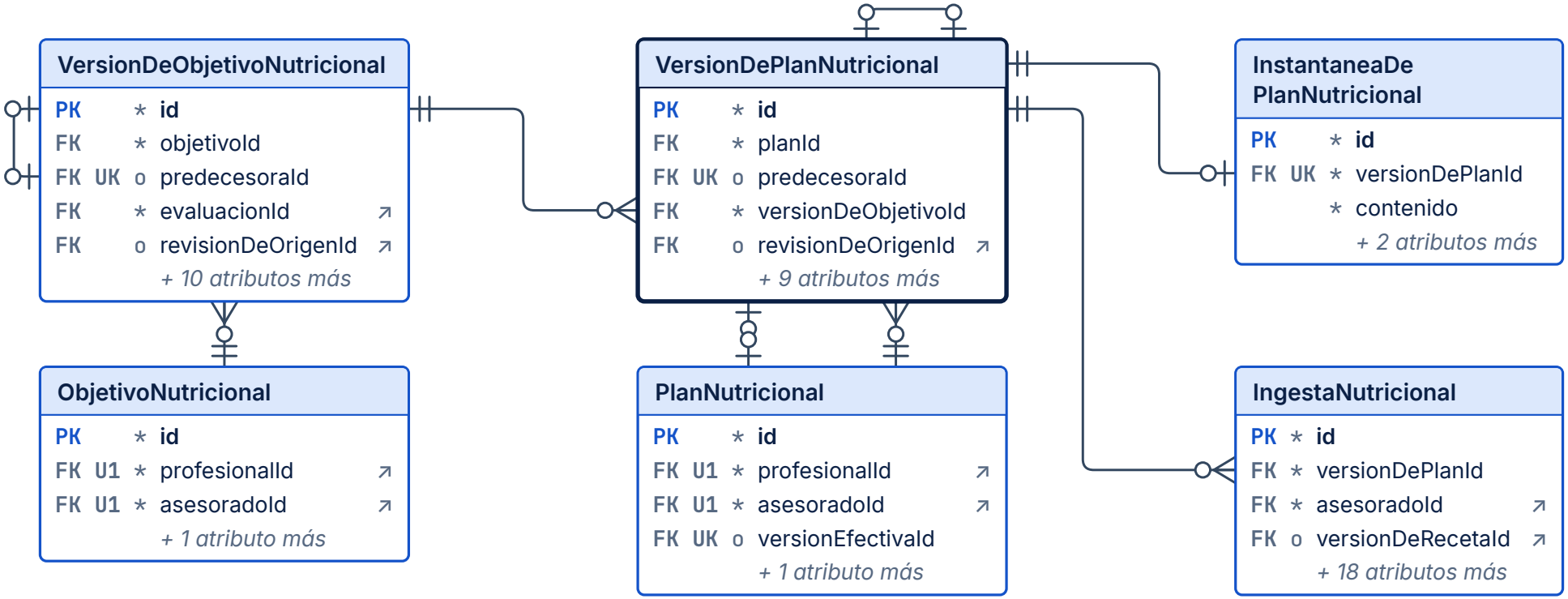
Qué representa. `vinculo`: la pareja profesional–persona (única: U1). `alcance_de_vinculo`: ese vínculo en un área, con su estado. `consentimiento`: la autorización de ese alcance; `version_de_consentimiento`: cada otorgamiento o revocación. `decision_de_acceso`: cada decisión del PDP.

Guardado o calculado. Se guardan los estados y cada decisión, con su resultado y las dimensiones que fallaron. El «modo de acceso» que muestran las listas (con acceso o bloqueado) se calcula en el momento.

Qué cambia y qué se conserva. Revocar cambia la situación y agrega una versión; finalizar cambia el estado del alcance. Nada se borra. Fuera del diagrama: `invitacion_de_vinculo` y `evento_de_vinculo`.

Nutrición: ¿cómo se guarda un plan sin que cambie después?

Nutrición El plan cambia por versiones; al activar, una instantánea congela la composición de cada alimento.



uno y solo uno · cero o uno · cero o muchos · PK FK claves · * obligatorio · o opcional · ↗ clave hacia otra área

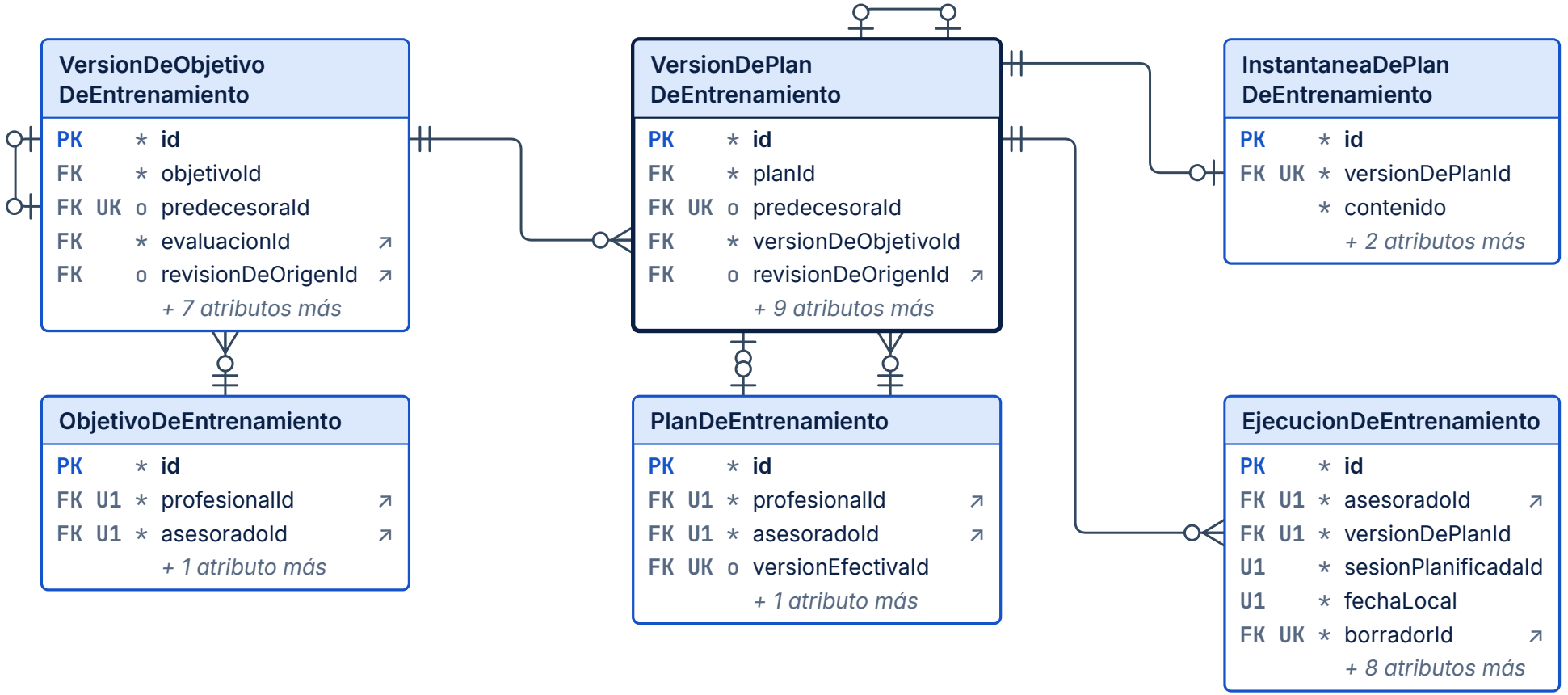
Qué representa. `plan_nutricional`: el plan de un profesional para una persona y su versión efectiva. `version_de_plan_nutricional`: un BORRADOR o una versión ACTIVADA, con días tipo, comidas y opciones en JSON. `instantanea_de_plan_nutricional`: la copia congelada al activar, con su huella. `ingesta_nutricional`: cada comida registrada.

Guardado o calculado. La comida guarda qué opción se registró y con qué cantidades. Las kcal y los macros **no** se guardan: se calculan cada vez con la instantánea. Lo indicado del día también se calcula.

Qué cambia y qué se conserva. Un borrador se edita; una versión activada no. Entre el plan y el catálogo no hay clave foránea: el contenido cita alimentos por id y la instantánea copia su composición. Por eso corregir el catálogo no reescribe un plan activado.

Entrenamiento: ¿qué se registra de una sesión?

Entrenamiento La misma idea que Nutrición: versiones, instantánea y una ejecución inmutable por sesión planificada.



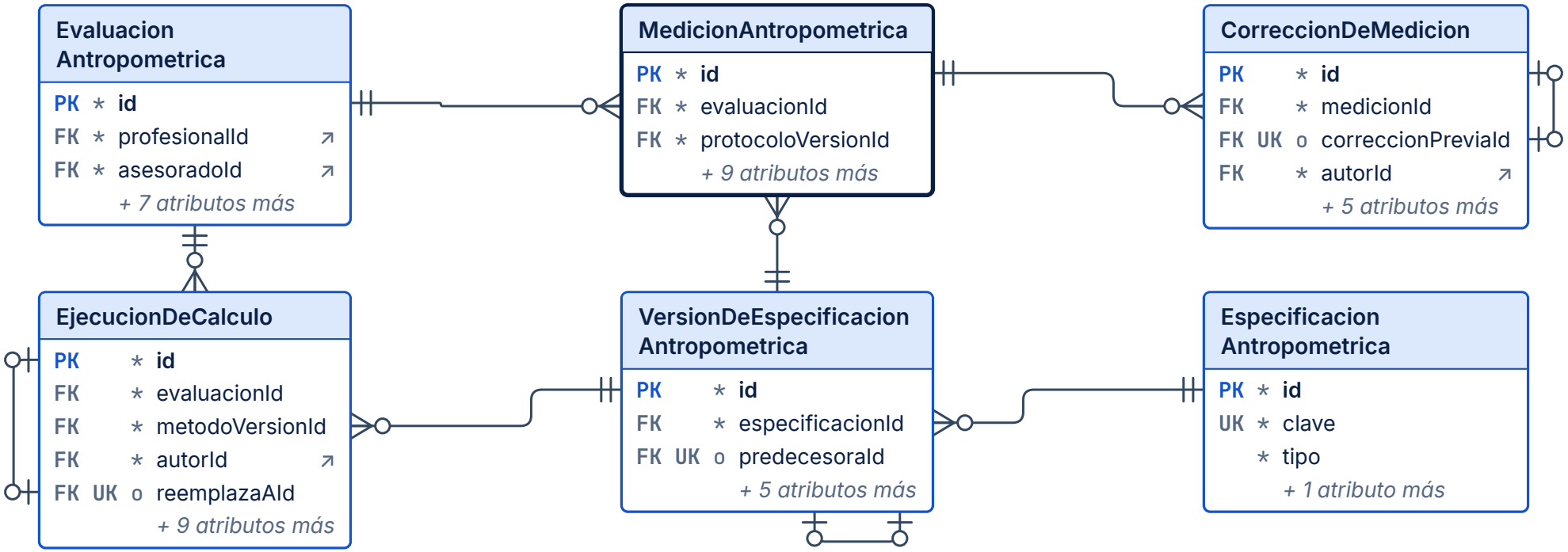
Qué representa. Objetivos y planes, con sus versiones (BORRADOR o ACTIVADA). `instantanea_de_plan_de_entrenamiento`: el orden y las relaciones tal como se emitieron. `ejecucion_de_entrenamiento`: la sesión registrada, una por ocurrencia.

Guardado o calculado. La ejecución guarda lo hecho (series, cargas, condición) en JSON y cuelga de la instantánea, nunca del catálogo. Mientras se entrena, se va guardando un borrador, que sí cambia.

Qué cambia y qué se conserva. La ejecución confirmada no cambia: corregirla agrega una fila de corrección. Fuera del diagrama: plantillas, evaluaciones, revisiones y eventos.

Antropometría: ¿por qué acá el resultado sí se guarda?

Antropometría Medidas directas, correcciones que no pisan, y cálculos guardados con todo lo necesario para repetirlos.



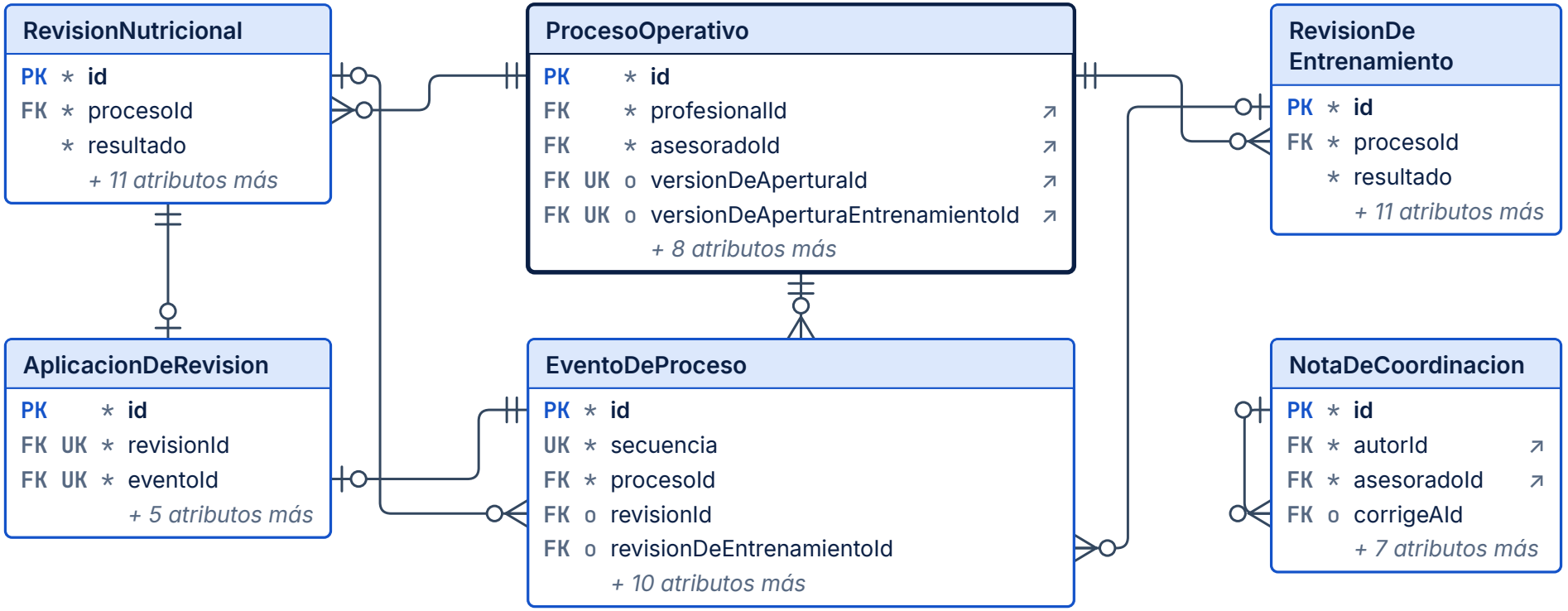
Qué representa. `evaluacion_antropometrica`: una toma. `medicion_antropometrica`: cada medida directa, con su unidad de origen. `correccion_de_medicion`: una cadena sobre la original, que no se toca. `especificacion_antropometrica` y su versión: la fórmula o el protocolo. `ejecucion_de_calculo`: un resultado.

Guardado o calculado. A diferencia de Nutrición, el resultado se guarda, con el método, su versión, las entradas efectivas y la precisión: así se puede decir con qué se calculó, aunque el método cambie. Que una medición esté vigente o anulada no es una columna: se deduce de si existe su anulación.

Qué cambia y qué se conserva. Publicar otra versión de una fórmula no reescribe cálculos viejos. Varias corridas conviven sin «ganador»; un recálculo apunta al anterior y no lo pisa.

Objetivos, procesos y revisiones: ¿cómo se ordena el seguimiento?

Objetivos, procesos y revisiones Un proceso abierto por profesional, persona y área; las revisiones y sus aplicaciones son hechos.



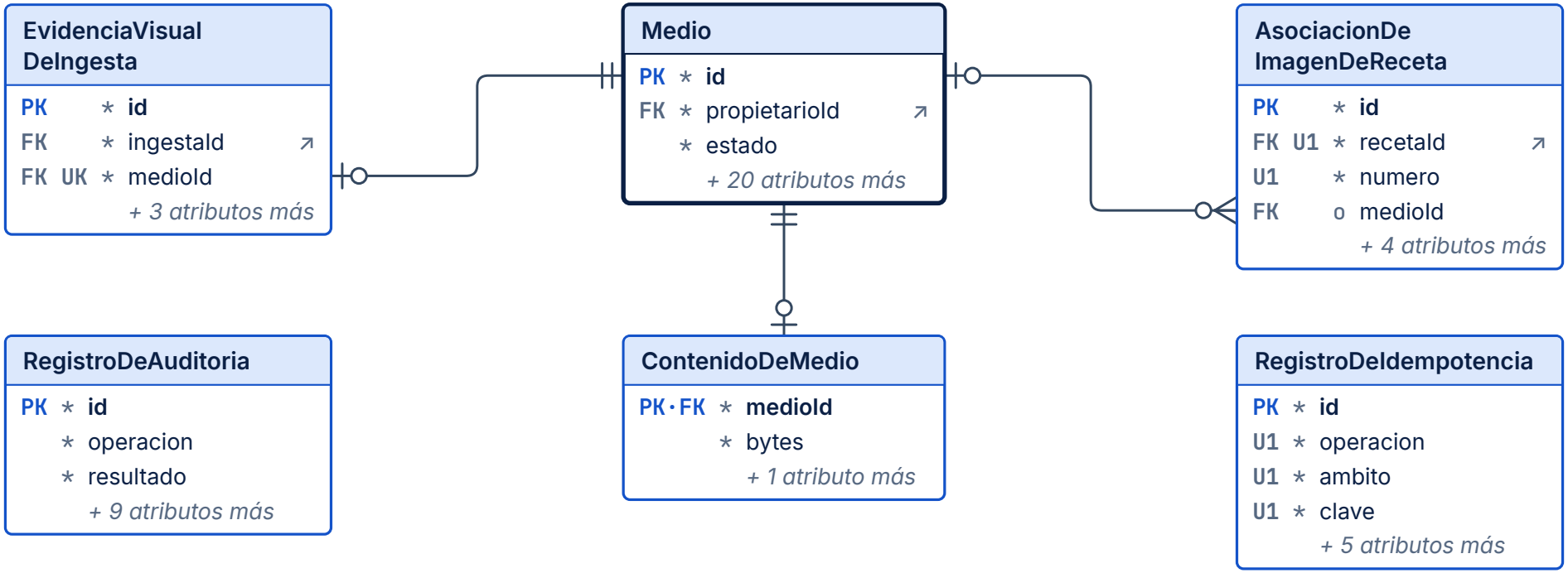
Qué representa. `proceso_operativo`: el seguimiento de un profesional con una persona en un área. `evento_de_proceso`: sus hechos. `revision_nutricional` y `revision_de_entrenamiento`: cada revisión, con su resultado. `aplicacion_de_revision`: aplicarla es otro hecho. `nota_de_coordinacion`: un mensaje entre los profesionales de una persona.

Guardado o calculado. Se guardan el estado del proceso (ABIERTO o CERRADO) y cada revisión completa. Que haya como máximo un proceso abierto por profesional, persona y área lo asegura un índice único parcial, no el código.

Qué cambia y qué se conserva. CERRADO es final: no se reabre. Las revisiones, las aplicaciones y las notas solo se agregan. Una nota no modifica planes, objetivos ni revisiones. Los objetivos de cada área están en sus páginas.

Auditoría, medios e idempotencia: ¿qué guarda BE sobre su propio funcionamiento?

Auditoría, medios y plataforma Fotos privadas, quién hizo qué y las respuestas guardadas para los reintentos.



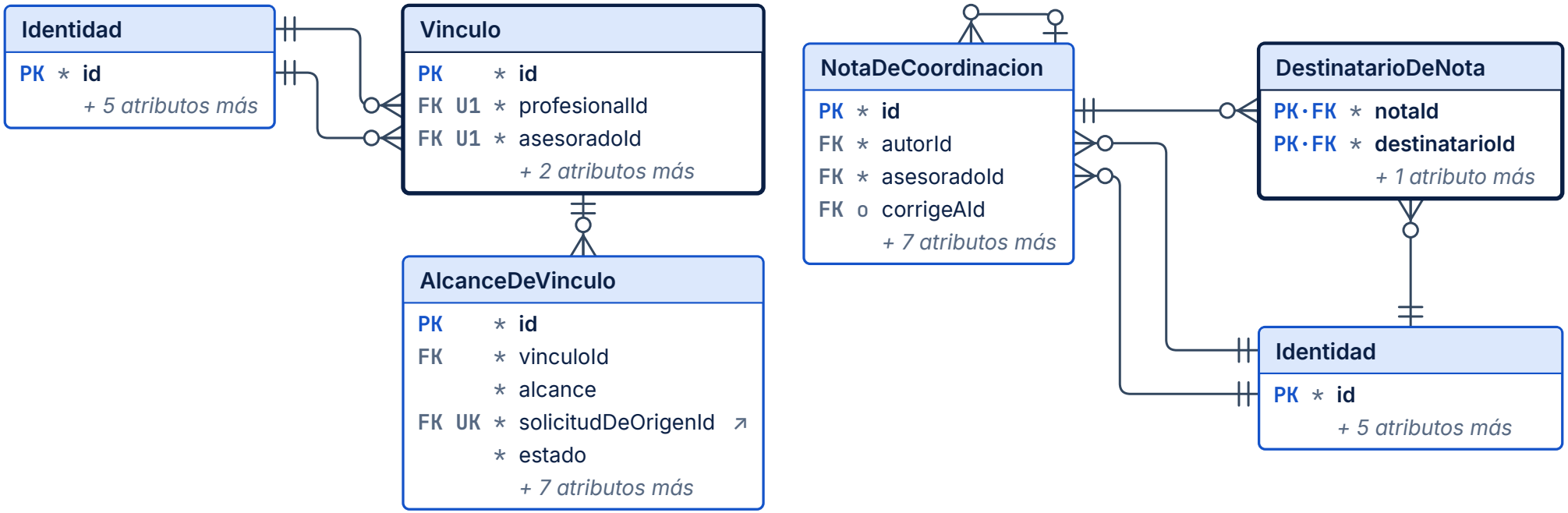
Qué representa. `medio`: los datos de una imagen; `contenido_de_medio`: sus bytes (`bytea`), en la misma base. `evidencia_visual_de_ingesta`: une una foto a una comida. `asociacion_de_imagen_de_receta`: cada cambio de imagen de una receta. `registro_de_auditoria` y `registro_de_idempotencia`.

Guardado o calculado. Se guardan los bytes ya procesados (JPEG). Los enlaces para verlos son temporales y firmados: no se guardan. La auditoría no guarda IP ni datos de salud; la idempotencia, una huella del pedido sin la credencial.

Qué cambia y qué se conserva. Suprimir una foto borra sus bytes y deja el registro del medio, con el momento y el motivo. Cambiar la imagen de una receta agrega una fila: la vigente es la de número más alto. Todo lo demás solo se agrega.

¿Cuándo una relación necesita su propia tabla?

Uno a muchos se resuelve con una columna; muchos a muchos, con una tabla en el medio. A veces esa tabla es un hecho del negocio.



Uno a muchos (1:N). Una versión de plan tiene muchas comidas registradas; cada comida, una sola versión. La clave foránea va del lado de «muchos»: `ingesta_nutricional.version_de_plan_id`.

Muchos a muchos con vida propia. Un profesional acompaña a muchas personas, y una persona a varios profesionales. `vinculo` y `alcance_de_vinculo` no solo los relacionan: tienen estado, versión e historia. Son hechos del negocio.

Muchos a muchos que solo relaciona. `destinatario_de_nota` une una nota con cada profesional que la recibe. Su clave primaria es compuesta (`nota_id`, `destinatario_id`) y no tiene datos propios, más allá del momento.

CONCEPTO 8

Cardinalidad

En criollo. Cuántos de un lado se relacionan con cuántos del otro: uno, cero o uno, muchos.
En BE. En los diagramas, la pata de gallo es «muchos» y la raya doble, «uno y solo uno».
Para comprobar. ¿Por qué la clave foránea de una relación 1:N va del lado de «muchos»?

CONCEPTO 9

Tabla intermedia

En criollo. La tabla que resuelve un muchos a muchos guardando un par por fila.
En BE. Si el par tiene estado o historia (`alcance_de_vinculo`) es una entidad; si no (`destinatario_de_nota`), es solo la relación.
Para comprobar. ¿Qué tendría que pasar para que `destinatario_de_nota` necesitara una columna de estado?

¿Qué impide que los datos queden inconsistentes?

Cuatro herramientas de la base, de la más simple a la más fuerte. Ninguna depende de que la API no se equivoque.

Herramienta	Qué asegura	En BE
Clave primaria	Que cada fila se pueda nombrar sin ambigüedad.	`id` UUID con `gen_random_uuid()`; compuesta en las tablas que solo relacionan.
Clave foránea	Que una referencia apunte a una fila que existe.	218 relaciones, todas con `Restrict`.
Único y único parcial	Que no haya dos filas iguales, o dos en cierta situación.	38 únicos parciales: «una raíz por cadena», «uno ABIERTO», «una comida efectiva».
Disparador	Reglas de historia y de estados que la base hace cumplir sola.	292 vigentes, en 27 de las 53 migraciones.

Un disparador real, completo en `prisma/migrations/20260919200100_vinculo_consentimiento_y_pdp`: la habilitación solo pasa de CONCEDIDA a RETIRADA o al revés, de a una versión, y no se borra.

```
-- Recortado: el alta, las columnas que no cambian y los códigos de error.
CREATE FUNCTION "be_habilitacion_guardar"() RETURNS trigger
  LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    RAISE EXCEPTION 'BE: una habilitación no se elimina (historia)';
  END IF;
  IF NEW."version" <> OLD."version" + 1 THEN
    RAISE EXCEPTION 'BE: la versión de la habilitación avanza de a uno';
  END IF;
  IF NOT ((OLD."estado" = 'CONCEDIDA' AND NEW."estado" = 'RETIRADA') OR
          (OLD."estado" = 'RETIRADA' AND NEW."estado" = 'CONCEDIDA')) THEN
    RAISE EXCEPTION 'BE: TRANSICION_NO_DECLARADA en habilitación % -> %',
      OLD."estado", NEW."estado";
  END IF;
  RETURN NEW;
END $$;
```

CONCEPTO 10

Integridad referencial

En criollo. Que toda referencia apunte a algo que existe. No puede haber una comida de un plan que no está.

En BE. Las 218 claves foráneas de BE usan `onDelete: Restrict`: no hay borrado en cascada. Para borrar algo referenciado habría que borrar antes lo que lo cita, y la historia no se borra.

Para comprobar. ¿Qué se perdería si borrar una versión de plan borrara en cascada sus comidas?

CONCEPTO 11

Restricción

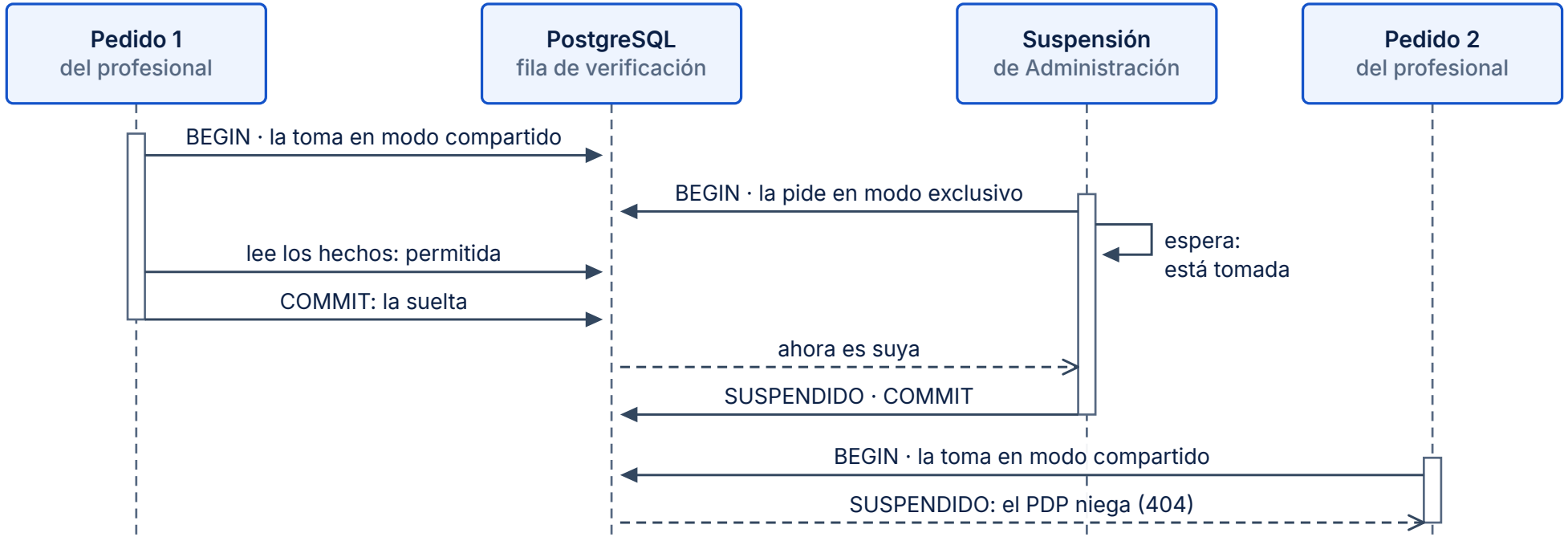
En criollo. Una regla que la base comprueba en cada escritura y que rechaza lo que no la cumple.

En BE. Unicidad (U1: un vínculo por pareja), unicidad parcial (un solo proceso ABIERTO por profesional, persona y área) y disparadores como el de la derecha.

Para comprobar. ¿En qué se diferencia un índice único parcial de un índice único común?

¿Qué pasa si algo falla a la mitad, o si dos pedidos llegan a la vez?

Si algo falla, la transacción se revierte entera. Si dos chocan, una espera a la otra, y la base decide el orden.



CONCEPTO 12

Atomicidad y reversión

En criollo. Una transacción ocurre entera o no ocurre. Si algo falla a la mitad, la base deshace todo (rollback).

En BE. Si una regla falla después del primer `INSERT`, no queda una comida a medias ni la auditoría de algo que no pasó.

Para comprobar. ¿Qué vería un profesional si se guardara una suspensión pero no su evento?

CONCEPTO 13

Bloqueo compartido y exclusivo

En criollo. Compartido: muchos leen la fila a la vez. Exclusivo: uno solo, y los demás esperan.

En BE. El PDP toma en modo compartido (`FOR SHARE`); la suspensión pide el exclusivo (`FOR NO KEY UPDATE`).

Para comprobar. ¿Por qué el PDP no toma la fila en modo exclusivo, si sería «más seguro»?

Cuando dos chocan igual. La idempotencia pone en fila los pedidos con la misma clave (`pg_advisory_xact_lock`). Si dos escriben lo mismo, un índice único decide: el segundo recibe un error y la API responde 409. Un interbloqueo se reintenta hasta tres veces y, si sigue, responde 409.

Un caso real. El PR #218 encontró dos configuraciones del autenticador que entraban juntas por tomar la fila en modo compartido: la segunda chocaba con un índice único y respondía 500. La corrección fue pedirla en modo exclusivo.

¿Para qué sirven los índices, y cómo guarda BE la historia?

Un índice hace rápida una búsqueda frecuente. La historia se guarda con cuatro mecanismos que nunca pisan una fila.

Índices que responden preguntas reales:

Índice	La pregunta que acelera
`ingesta_nutricional (asesorado_id, fecha_local)`	¿Qué registró esta persona entre estas fechas?
`ingesta_nutricional (version_de_plan_id)`	¿Qué se registró contra esta versión del plan?
`decision_de_acceso (actor_id, sujeto_id, alcance, momento_de_registro)`	¿Qué decidió el PDP para este profesional y esta persona?
Único parcial `ingesta_nutricional_prescripta_unica`	No acelera: impide dos comidas efectivas iguales.

Cuatro formas de guardar la historia:

Mecanismo	Cómo funciona	Ejemplo
Hechos	Una fila por cosa que pasó, en orden (`secuencia`).	`evento_de_nutricion`, `decision_de_acceso`
Versiones	Cada cambio, una fila que apunta a su predecesora; un índice asegura una sola raíz.	`version_de_plan_nutricional`, `version_de_consentimiento`
Instantáneas	Una copia fija al activar, con una huella SHA-256 para detectar cambios.	`instantanea_de_plan_nutricional`
Correcciones	Una fila nueva que dice qué corrige; la original no se toca.	`rectificacion_de_cantidades`, `anulacion_de_ingesta`

Y la estructura misma tiene historia: las 53 migraciones (guía central, nivel 5).

CONCEPTO 14

Índice

En criollo. Como el índice de un libro: una lista ordenada que lleva directo a las filas, sin leer toda la tabla.
En BE. «Analizar» pide las comidas de una persona en un período: el índice (`asesorado_id`, `fecha_local`) de `ingesta_nutricional` va derecho a esas filas.
Para comprobar. Si un índice acelera las lecturas, ¿por qué no poner uno en cada columna?

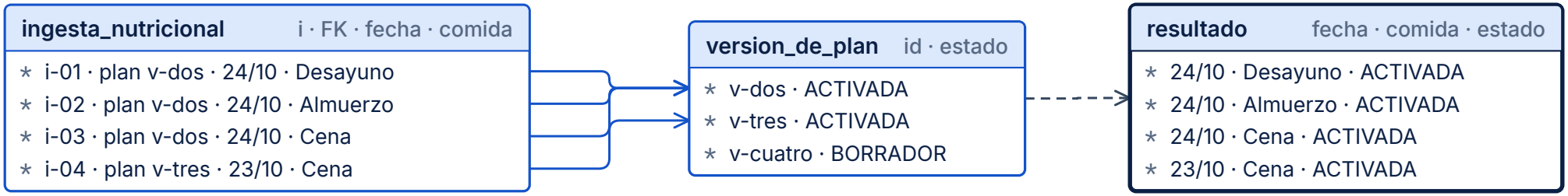
CONCEPTO 15

Solo agregar (append-only)

En criollo. Una tabla en la que se agregan filas y nunca se modifican ni se borran.
En BE. `be_solo_agregar()` rechaza todo `UPDATE` o `DELETE` en las tablas de hechos, como `registro_de_auditoria` o `evento_de_nutricion`.
Para comprobar. Si nada se modifica, ¿cómo se corrige una comida mal cargada?

¿Cómo se juntan dos tablas en una consulta?

Con un JOIN, por la clave foránea. Las consultas de estas dos páginas solo leen, con identificadores inventados.



Leído en criollo

«Para cada comida registrada, traeme la versión del plan que cita, y mostrame las dos cosas juntas». La versión v-cuatro no aparece en el resultado: ninguna comida la cita (un INNER JOIN deja afuera lo que no empareja; un LEFT JOIN la conservaría con las columnas de la otra tabla vacías).

```sql
-- 1. JOIN: las comidas de octubre con la versión del
--    plan que regía, sin las anuladas.
SELECT i.fecha_local, i.comida_id, v.version
  FROM ingesta_nutricional i
  JOIN version_de_plan_nutricional v
    ON v.id = i.version_de_plan_id
  LEFT JOIN anulacion_de_ingesta a
    ON a.ingesta_id = i.id
 WHERE i.asesorado_id = '00000000-0000-4000-8000-000000000001'
   AND i.fecha_local BETWEEN '2026-10-01' AND '2026-10-31'
   AND a.id IS NULL      -- sin anulación
 ORDER BY i.fecha_local;
```

```sql
-- 2. LEFT JOIN: los vínculos de un profesional en
--    Nutrición, con su autorización si la hay.
SELECT av.id, av.estado, c.situacion
  FROM vinculo vi
  JOIN alcance_de_vinculo av ON av.vinculo_id = vi.id
                            AND av.alcance = 'NUTRICION'
  LEFT JOIN consentimiento c
    ON c.alcance_de_vinculo_id = av.id
 WHERE vi.profesional_id = '00000000-0000-4000-8000-000000000002';
```

Sin el `LEFT JOIN`, un vínculo que todavía no tiene autorización desaparecería del resultado: el `JOIN` común solo deja las filas que tienen pareja del otro lado. La 1 usa el mismo truco al revés: pide las comidas **sin** anulación (`a.id IS NULL`).

¿Cómo se cuenta y se agrupa, sin inventar ceros?

GROUP BY junta las filas que comparten un valor y las cuenta. Lo que no tiene filas no aparece: no es un cero.

```sql
-- 3. Por qué se negaron los pedidos de un profesional
--    en la última semana.
SELECT d.operacion, dim AS dimension, count(*) AS veces
  FROM decision_de_acceso d,
       unnest(d.dimensiones_desfavorables) AS dim
 WHERE d.actor_id = '00000000-0000-4000-8000-000000000002'
   AND d.resultado = 'DENEGADA'
   AND d.momento_de_registro > now() - interval '7 days'
 GROUP BY d.operacion, dim
 ORDER BY veces DESC;
```

```sql
-- 4. Comidas registradas por día. Un día sin registros
--    NO aparece: no es un cero.
SELECT i.fecha_local, count(*) AS comidas
  FROM ingesta_nutricional i
 WHERE i.asesorado_id = '00000000-0000-4000-8000-000000000001'
 GROUP BY i.fecha_local
 ORDER BY i.fecha_local;
```

«0 comidas registradas» no quiere decir «no comió»: quiere decir «no registró». Por eso «Analizar» muestra un día sin registros como faltante y no como cero.

Cómo se lee una consulta. Se escribe en un orden y se ejecuta en otro. PostgreSQL arma primero las filas (`FROM` y los `JOIN`), después las filtra (`WHERE`), las agrupa (`GROUP BY`), elige las columnas (`SELECT`) y al final las ordena (`ORDER BY`). Por eso `WHERE` no puede usar un `count(*)`: cuando filtra, todavía no se contó nada; para filtrar grupos está `HAVING`.

CONCEPTO 16

JOIN y LEFT JOIN

En criollo. JOIN deja solo las filas con pareja. LEFT JOIN deja todas las de la izquierda, con vacíos (NULL) donde no hay pareja.
En BE. La consulta 2 muestra los vínculos aunque no tengan autorización: `c.situacion` viene en NULL.
Para comprobar. ¿Qué cambia en la consulta 1 si el `LEFT JOIN` de la anulación pasa a ser un `JOIN`?

CONCEPTO 17

Agrupar (GROUP BY)

En criollo. Juntar las filas que comparten un valor para contarlas o sumarlas por grupo.
En BE. Comidas por día, o decisiones denegadas por dimensión. Un día sin comidas no tiene filas que agrupar.
Para comprobar. ¿Cómo harías para que la consulta 4 muestre también los días sin registros, y qué dirías de ellos?

Solo lectura

Las cuatro consultas leen. En BE nadie escribe en la base a mano: las escrituras pasan por la API, que valida, decide el permiso y audita. Incluso en test, una consulta directa necesita abrir una IP por un rato, y se cierra al terminar.

¿En qué se diferencia el modelo conceptual del físico?

El conceptual dice qué cosas existen y cómo se relacionan; el físico, cómo se guardan. Este cuaderno muestra los dos, separados.

	Conceptual	Físico (PostgreSQL)
Nombres	Persona, Plan, Comida registrada	`identidad`, `version_de_plan_nutricional`, `ingesta_nutricional`
Relaciones	«Una comida se registra contra una versión del plan»	La columna `version_de_plan_id`, clave foránea con `Restrict`
Identidad	Cada cosa es distinguible	`id` UUID generado por la base
Detalle de un plan	Días tipo, comidas y opciones	Una columna JSON en la versión, validada por el contrato zod
Reglas	«Lo registrado no se pisa»	Disparador `be_solo_agregar` e índices únicos parciales
Tiempo	«La comida del martes»	`fecha_local` (DATE), `zona_horaria` y `momento_de_ocurrencia` (con zona)

Lo que este cuaderno dejó afuera, a propósito. Los diagramas de este cuaderno muestran 43 de las 118 tablas y, de cada una, solo las claves y algunos atributos («+ N atributos más»). El resto está en las 15 vistas del DER de la tesis (definidas en `laminas/motor/vistas-der.mjs`) y en `prisma/schema.prisma`, que es la fuente.

Una advertencia al leer los diagramas
Un diagrama por área muestra todas las relaciones entre las tablas elegidas, pero no las que salen hacia tablas que no están en la vista: esas se marcan con una flecha (↗) en la columna que las tiene.

CONCEPTO 18

Modelo conceptual

En criollo. Las cosas del negocio y sus relaciones, sin decidir todavía cómo se guardan.
En BE. «Una persona registra comidas contra la versión de un plan»: tres conceptos y dos relaciones.
Para comprobar. ¿Por qué conviene discutir primero el conceptual con un profesional de la salud?

CONCEPTO 19

Modelo físico

En criollo. Las tablas, columnas, tipos, claves, índices y disparadores con que eso se guarda en un motor concreto.
En BE. `ingesta_nutricional` con `version_de_plan_id` (UUID), `fecha_local` (DATE), cantidades en JSON, dos índices y el disparador que la deja solo agregar.
Para comprobar. Nombrá una decisión física de BE que no cambia nada del modelo conceptual.